

# Felügyelt finomhangolású Gemini a prediktív karbantartáshoz

[vertex-ai-platform](#)[google-cikk](#)[vertex-ai-vezír](#)[vertex-ai-tenzortábla](#)[vertex-ai-kísérletek](#)[aniketagrawal Google-alkalmazottak](#)

1d

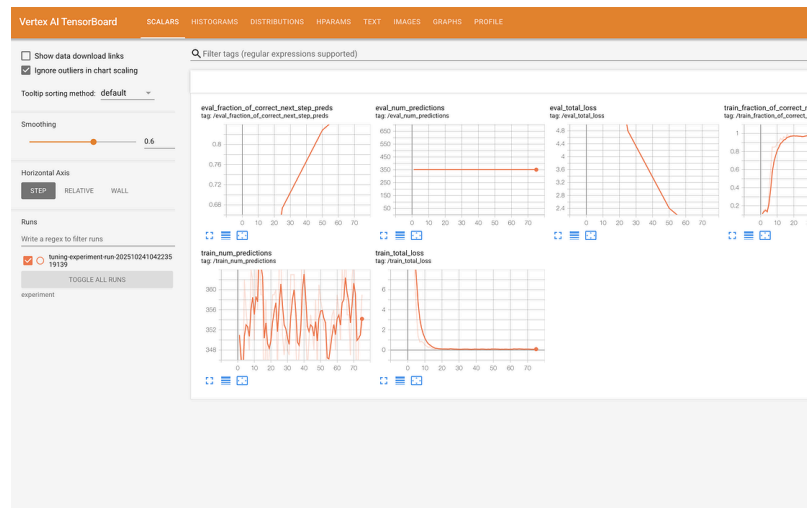
## Kovácsolási specialista mesterséges intelligencia: Kódolók útmutatója a finomhangoláshoz

Az olyan alapmodellek, mint a Gemini, a mesterséges intelligencia forradalmának alapanyagát jelentik – erős, generalista intellektusokat. De a nyersanyag nem old meg konkrét üzleti problémákat. A valós érték az, ha ezeket a generalistákat specialistákká kovácsoljuk: egy jogi MI, amely érti a szerződési jogot, egy pénzügyi MI, amely felismeri a kifinomult csalásokat, egy gyártási MI, amely képes meghallani a közelgő gépmeghibásodás finom jeleit.

Hogyan építed fel ezt a mesterséges intelligencia gyárat? Hogyan hozol létre egy olyan folyamatot, amely nem csupán egy egyszeri tudományos kísérlet, hanem egy megismételhető, szabályozott és skálázható mérnöki tudományág?

Ez a **Vertex AI** tervrajza .

[Ugrás a fő tartalomra](#)



Ez az útmutató egy átfogó, architekturális áttekintést nyújt a Google egységes mesterséges intelligencia platformjának használatáról, amely segítségével szisztematikusan hozunk létre speciális modelleket. Illusztrációs szálként egy valós, nagy értékű problémát – egy „digitális technikus” felépítését – fogunk használni. Részletesen elemezzük az egyes Vertex mesterséges intelligencia képességeket, közvetlenül a munkafüzetünkből származó, erősen annotált kódot biztosítva, és elmagyarázva azokat a stratégiai mérnöki döntéseket, amelyek lehetővé teszik a sebességet, az irányítást és a skálázhatóságot.

## Az ügyfélút: az MLOps káosztól az egységes mesterséges intelligenciagyárig

Képzeld el Alexet, egy vezető gépi tanulási platformmérnököt.

**Az „előtte” állapot (a fájdalom):** Alex világa a működési súrlódások széttöredezett tájképe.

- **Adatsilók** : Az érzékelőadatok egy adattörténeaszben, a naplók az Elasticsearchben található, a betanítási adatok pedig véletlenszerű CSV-fájlokként kerülnek egy GCS-vödörbe.



- **Elszigetelt fejlesztés** : Az adatkutatók helyi Jupyter jegyzetfüzetekben dolgoznak. A kódjuk nincs verziózva, és a környezetük nem reprodukálható.
- **Manuális telepítési pokol** : Egy modell telepítéséhez Alexnek manuálisan kell konténerbe helyeznie azt Dockerrel, egyéni Flask API-t kell írnia, konfigurálnia kell egy Kubernetes fürtöt automatikus skálázással, és be kell állítania egy terheléelosztót. Ez egy kéthetes folyamat minden új modellverzió esetében.
- **Nincs irányítás** : Amikor egy éles modell teljesítménye romlik, nincs auditnapló. Melyik adathalmazon képezték ki? Mik voltak a hiperparaméterek? Senki sem tudja biztosan.

**Az „utána” állapot (A vízió):** Alex egy új munkafolyamatot fog megtervezni teljes egészében a Vertex AI-ra alapozva, létrehozva egy valódi AI-gyárat.

- **Egységes adatforrás** : Minden adat egy verziózott, biztonságos GCS-tárolóban tárolódik, amely az igazság egyetlen forrásaként működik.
- **Kiszolgáló nélküli specializáció** : Az alapmodell hatékonyan finomhangolható egy felügyelt, kiszolgáló nélküli betanítási szolgáltatással. Nincsenek GPU-k kezelését igénylő megoldások.
- **Automatizált irányítás** : Minden elem, metrika és művelet automatikusan nyomon követésre, verziózásra és összekapcsolásra kerül egy lekérdezhető leszármazási gráfban.
- **Egykattintásos telepítés** : A specializált modell egyetlen paranccsal telepíthető egy biztonságos, automatikusan skálázódó végpontra.



Egy éles üzemű rendszerhez szilárd alapokra van szükség. Ez a szakasz a notebookunk kezdeti beállítási celláit tükrözi.

## 0.1. Projekt és API konfiguráció

1. **Google Cloud Project** : Szüksége van egy olyan Google Cloud projektre, amelyen engedélyezve van a számlázás.
2. **API-k engedélyezése** : Győződjön meg arról, hogy a következő API-k engedélyezve vannak: `aiplatform.googleapis.com` (Vertex AI) és `storage.googleapis.com` (Cloud Storage).
3. **Engedélyek (IAM)** : A felhasználói fiókjához vagy szolgáltatásfiókjához `Vertex AI User`, `Storage Admin` és `Service Account User` szerepkörökre van szükség.

## 0.2. Fejlesztői környezet és hitelesítés

Egy **Vertex AI Workbench** példány használata erősen ajánlott. A következő, közvetlenül a jegyzetfüzetünkéből vett kód kezeli a hitelesítést és a konfigurációt.

```
import os
import google.generativeai as genai
import vertexai
from google.genai import Client as VertexGenAI

# --- Vertex AI Configuration ---
PROJECT_ID = "" # @param {type: "string"}
REGION = "" # @param {type: "string"}
BUCKET_NAME = "" # @param {type: "string"}
BUCKET_URI = f"gs://{BUCKET_NAME}"

# --- Authentication ---
# This block handles authentication, which is
# It attempts to get the project ID from the
if not PROJECT_ID:
    from google.colab import auth
    auth.authenticate_user()
    import subprocess
    PROJECT_ID = (
        subprocess.check_output(["gcloud", "c
        .decode("utf-8")
```



```

        .strip()
    )

    # --- GCS Bucket Creation ---
    # This command ensures the GCS bucket exists,
    # It's an idempotent operation, safe to run m
    creation_command = f"gsutil ls {BUCKET_URI} > /dev/null"
    os.system(creation_command)

    # --- SDK Initialization ---
    # This is a critical step. It configures all
    # specific project, region, and staging bucket
    vertexai.init(project=PROJECT_ID, location=REGION)

    # The google-genai library has two modes. We
    # configured for the Vertex AI backend for pl
    vertex_client = VertexGenaiLowLevelClient(
        vertexai=True, project=PROJECT_ID, location=REGION
    )

```



### 0.3. Globális konstansok

A konstansok definiálása a jegyzetfüzet elején biztosítja a konzisztenciát és elkerüli a mágikus sztringeket.

```

# The specific, tunable version of the Gemini
BASE_MODEL_ID = "gemini-2.0-flash-001"

# A dynamic name for our tuned model to ensure uniqueness
TUNED_MODEL_DISPLAY_NAME = f"pred-maint-gemini-2.0-flash-001"

# GCS paths for our datasets, derived from the bucket URI
DATA_DIR_GCS = f"{BUCKET_URI}/pred_maint_tuned_model_data"
TRAIN_JSONL_GCS_URI = f"{DATA_DIR_GCS}/train_data.jsonl"
VALIDATION_JSONL_GCS_URI = f"{DATA_DIR_GCS}/validation_data.jsonl"
TEST_JSONL_GCS_URI = f"{DATA_DIR_GCS}/test_data.jsonl"

```



## 1. rész: Az Adataalapítvány – Natív integráció a felhőalapú tárhellyel

[Ugrás a fő tartalomra](#)



A Vertex AI egy adatközpontú munkafolyamat köré épül, amelynek középpontjában a **Google Cloud Storage (GCS)** áll.

### 1.1. Képességfókuszban: A GCS mint mesterséges intelligencia adattár

- **Közvetlen integráció** : A Vertex AI szolgáltatások közvetlenül a GCS URI-kból olvasnak `gs://...`, kiküszöbölve ezzel az összetett adatbetöltőket.
- **Teljesítmény** : A GCS nagy áteresztőképességű hozzáférést biztosít, ami elengedhetetlen a GPU-k I/O szűk keresztmetszetek nélküli betáplálásához.
- **Verziókezelés** : A GCS objektumok verziókezelése megváltoztathatatlan előzményeket hoz létre az adathalmazokról, ami a reprodukálhatóság sarokköve.

### 1.2. A JSONL formátum: A hangolás nyelve

**Az adatainkat JSON Lines (JSONL)** formátumban kell előkészítenünk . Egyetlen hatalmas JSON tömbbel ellentétben ez a formátum **streamelhető** . A hangolási szolgáltatás soronként képes olvasni és feldolgozni a fájlt, így terabájtos méretű adathalmazokat is képes kezelni anélkül, hogy a teljes fájlt a memóriába kellene tölteni.

### 1.3. Kód működés közben: Adatátalakítás és feltöltés

A notebook `create_tuning_jsonl` függvénye elvégzi a nyers szenzoradatok strukturált, gyors válasz formátumba történő átalakítását. A következő függvény ezután feltölti ezeket az adatokat.

```
def save_jsonl_to_gcs(instances: list[dict],  
    """  
    Serializes a list of Python dictionaries  
    This function bridges the gap between our
```



```

"""
if not instances:
    print(f"No instances to upload to {gcs}")
    return

# 1. Parse the GCS URI to get the bucket
parts = gcs_uri[5:].split("/", 1)
bucket_name = parts[0]
blob_name = parts[1]

# 2. Convert each Python dictionary into jsonl_content = "\n".join([json.dumps(instance) for instance in instances])

# 3. Instantiate the GCS client and upload
storage_client = storage.Client(project=PROJECT_ID)
bucket = storage_client.bucket(bucket_name)
blob = bucket.blob(blob_name)
blob.upload_from_string(jsonl_content, content_type="application/json")
print(f"Uploading {len(instances)} instances to GCS")

# The main script calls this for each data split
# save_jsonl_to_gcs(train_split, TRAIN_JSONL_URI)
# save_jsonl_to_gcs(validation_split, VALIDATION_JSONL_URI)
# save_jsonl_to_gcs(test_split, TEST_JSONL_URI)

```

|                      |                                                                                                                                  |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Cloud Storage        | Bucket details                                                                                                                   |
| Overview             | diwali_bucket                                                                                                                    |
| Buckets              | Location: us-central1 (Iowa)   Storage class: Standard   Public access: Subject to object ACLs   Protection: Soft Delete         |
| Monitoring           | Objects   Configuration   Permissions   Protection   Lifecycle   Observability <span>New</span>   Inventory Reports   Operations |
| Settings             |                                                                                                                                  |
| Storage Intelligence |                                                                                                                                  |
| Insights datasets    |                                                                                                                                  |
| Configuration        |                                                                                                                                  |

| Buckets > diwali_bucket > pred_maint_tuning_data                 |          |                   |                           |               |               |  |
|------------------------------------------------------------------|----------|-------------------|---------------------------|---------------|---------------|--|
| Create folder   Upload   Transfer data   Other services          |          |                   |                           |               |               |  |
| Filter by name prefix only   Filter   Filter objects and folders |          |                   |                           |               |               |  |
| Name                                                             | Size     | Type              | Created                   | Storage class | Last modified |  |
| test_data.jsonl                                                  | 84.7 KB  | application/jsonl | Oct 24, 2025, 11:10:49 PM | Standard      | Oct 24, 2025  |  |
| train_data.jsonl                                                 | 676.7 KB | application/jsonl | Oct 24, 2025, 11:10:48 PM | Standard      | Oct 24, 2025  |  |
| validation_data.jsonl                                            | 84.6 KB  | application/jsonl | Oct 24, 2025, 11:10:48 PM | Standard      | Oct 24, 2025  |  |

## 2. rész: A hangolómotor – Szerver nélküli, hatékony specializáció

A **Vertex AI Training** szolgáltatás a mesterséges intelligencia gyárunk központi motorja.

### 2.1. A szerver nélküli paradigma: Az infrastruktúra absztrahálása

When you launch a tuning job, you **do not** manage VMs, GPUs, networks, or clusters. You submit a job



definition, and Vertex AI's fleet management system handles provisioning, orchestration, fault tolerance (restarting from checkpoints if a node fails), and tear-down. This transforms a complex infrastructure problem into a simple API call.

## 2.2. The “Why” of PEFT/LoRA: A Deep-Dive into Efficient Specialization

Full fine-tuning is slow, expensive, and risks “catastrophic forgetting.” **PEFT (Parameter-Efficient Fine-Tuning) via LoRA (Low-Rank Adaptation)** is the solution. It freezes the base model's weights and injects small, trainable “adapter” matrices. We only train these adapters, which represent the *delta* for the new task. On Vertex AI, this state-of-the-art technique is enabled with a single parameter.

## 2.3. Code in Action: Launching the Tuning Job

The Vertex AI SDK provides a declarative interface. We define *what* we want, and the platform figures out *how*.

```
# This code is from the notebook's "Step 4: L
TUNING_JOB_NAME = None
if not train_split or not validation_split:
    print("Skipping fine-tuning job launch as
else:
    # 1. Define the datasets using the GCS UR
    training_dataset = {"gcs_uri": TRAIN_JSON
    validation_dataset = genai_types.TuningVa

    # 2. Define the tuning configuration.
    tuning_config = genai_types.CreateTuningJ
    # This is the key to enabling PEFT/Lo
    # controlling the capacity of the ada
    adapter_size="ADAPTER_SIZE_FOUR",

    # An epoch is one full pass over the
    epoch_count=3,

    # This name registers the resulting m
    tuned_model_display_name=TUNED_MODEL_
```

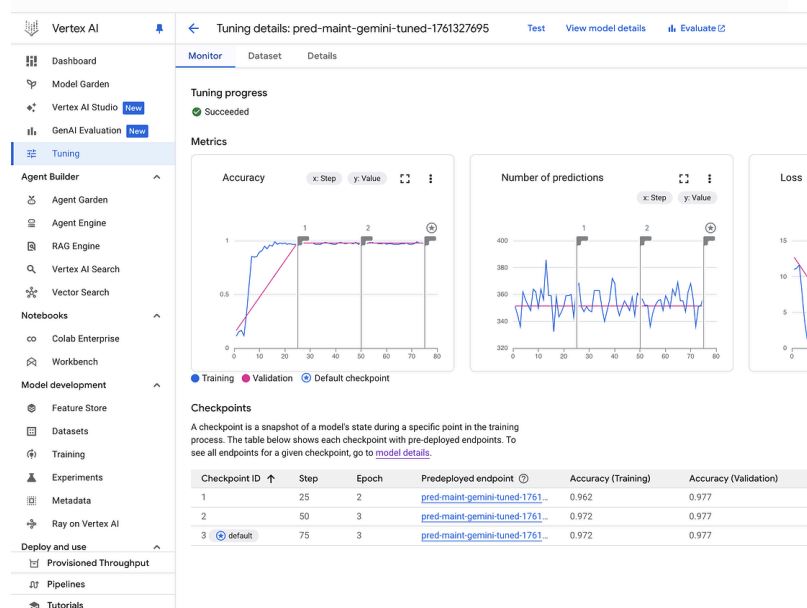
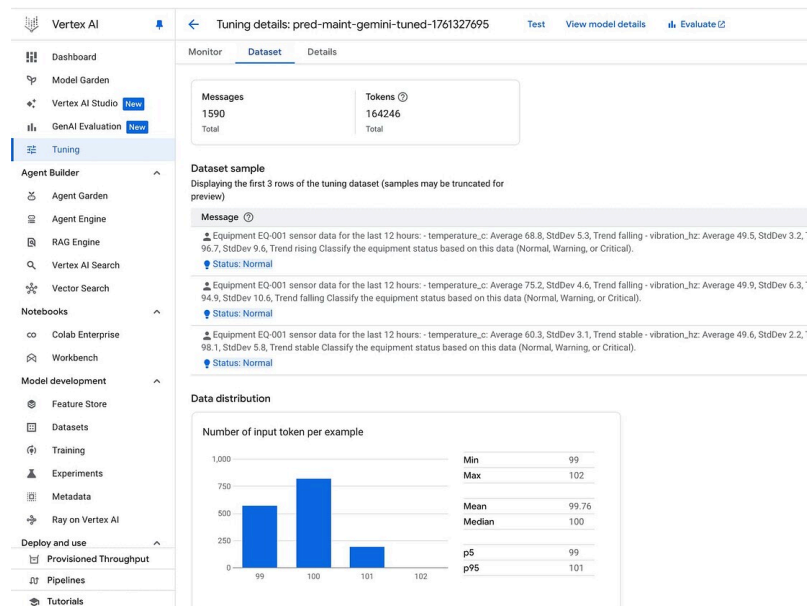




```
# The validation dataset provides an
validation_dataset=validation_dataset
)

# 3. Launch the serverless tuning job. Th
sft_tuning_job = vertex_client.tunings.tu
base_model=BASE_MODEL_ID,
training_dataset=training_dataset,
config=tuning_config,
)

TUNING_JOB_NAME = sft_tuning_job.name
print(f"\nTuning job created: {sft_tuning
```

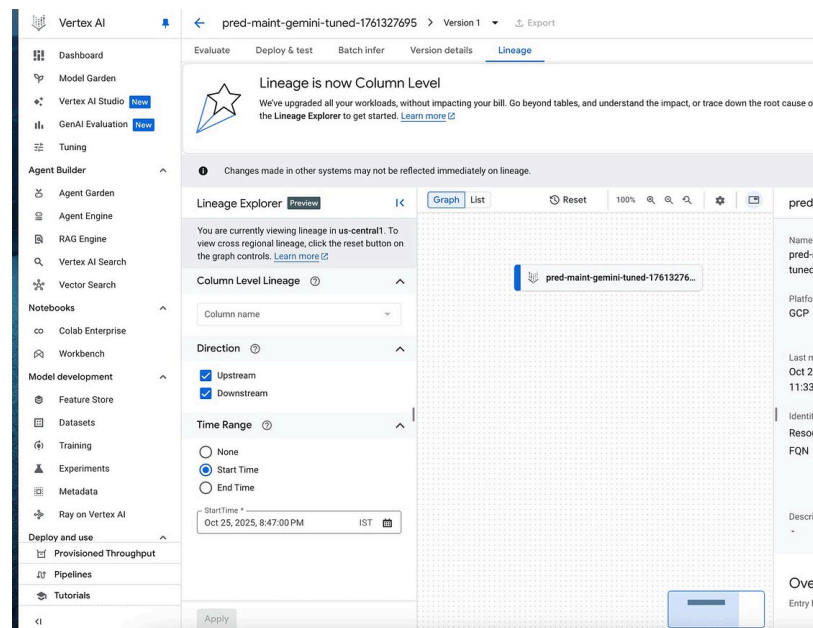


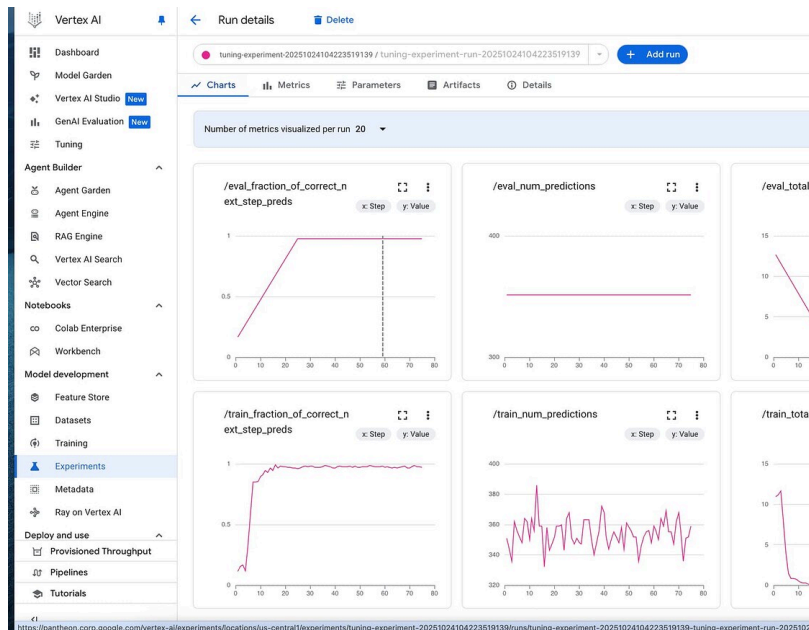
## Part 3: The Governance Layer - Automatic Tracking and Auditing

A model without a history is a liability. Vertex AI automatically provides governance through three interconnected services.

### Capability Spotlight: Experiments, Lineage, and Model Registry

1. **Vertex AI Experiments:** Every tuning job is automatically logged as an experiment run, capturing hyperparameters, data sources, and metrics.
2. **Vertex AI Model Registry:** The tuned model is automatically versioned and registered here, becoming the canonical, deployable artifact.
3. **Vertex AI Lineage:** This is the crown jewel of MLOps. It automatically creates a graph connecting the data source, the training job, and the resulting model artifact, providing an unbreakable and automated audit trail.





## Part 4: The Deployment Engine - Scalable, Secure Endpoints

Once our model is in the Model Registry, deployment is a simple, robust process.

### Capability Spotlight: Vertex AI Endpoints

- **Egykattintásos telepítés** : Telepítse modelljét egy biztonságos HTTPS végpontra egyetlen SDK-hívással.
- **Kiszolgáló nélküli és skálázható** : A végpont automatikusan skálázódik a forgalom alapján, nullától több ezer RPS-ig.
- **Integrált monitorozás** : A végpontok beépített monitorozással rendelkeznek a forgalom, a hibaszázalék és a késleltetés tekintetében.
- **Forgalomfelosztás** : Az új verziók biztonságos bevezetése A/B tesztelés és canary deployments segítségével.

### Kód működés közben: A kiküldött szakember értékelése

`evaluate_qualitatively` A jegyzetfüzetben található függvény bemutatja, hogyan lehet lekérdezni az új



- ❶ [A fórumokról](#)
- ❷ [Közösségi irány...](#)
- ➔ [Első lépések](#)
- 👥 [Csoportok](#)
- ⋮ [Több](#)
- ▼ [Kategóriák](#)
- ☰ [Minden kategória](#)
- ▼ [Címkék](#)
- 🔍 [google-...](#) 1 olvasa...
- ☰ [Minden címke](#)

+ 

végpontot .

```
# This code is from "Step 6: Evaluate Tuned M
def evaluate_qualitatively(tuned_endpoint: st
    """Makes predictions with the tuned model
    if not tuned_endpoint:
        print("Tuned model endpoint not avail
        return

    # 1. Select a random, unseen sample from
    sample = random.choice(test_data)
    user_prompt = sample["contents"][0]["part
    expected_output = sample["contents"][1]["

    # 2. Prepare the prediction request. It o
    prediction_contents = [{"role": "user", "

    # 3. Call the endpoint using the Vertex A
    # Note: The 'model' argument is the full
    response = vertex_client.models.generate_
        model=tuned_endpoint,
        contents=prediction_contents,
        config={
            # Temperature controls randomness
            # likely, factual answer, so we s
            "temperature": 0.1,
            "max_output_tokens": 50,
        },
    )

    predicted_output = response.text.strip()
    print(f"Input Prompt:\n{user_prompt}")
    print(f"\nExpected Output: {expected_outp
    print(f"Predicted Output: {predicted_outp
    print(f"Result: {'MATCH' if predicted_out

# The main script waits for the tuning job to
# TUNED_MODEL_ENDPOINT = sft_tuning_job.tuned
# evaluate_qualitatively(TUNED_MODEL_ENDPOINT
```



[Ugrás a fő tartalomra](#)



Vertex AI

Dashboard

Model Garden

Vertex AI Studio

GenAI Evaluation

Tuning

Agent Builder

Agent Garden

Agent Engine

RAG Engine

Vertex AI Search

Vector Search

Notebooks

Colab Enterprise

Workbench

Model development

Feature Store

Datasets

Training

Experiments

Metadata

Ray on Vertex AI

Deploy and use

Provisioned Throughput

Pipelines

Tutorials

Experiments

Experiment trackingVizier studiesTensorboard instances

Vertex AI Experiments lets you track and compare machine learning experiments, which you can share with others.

To get started, create an experiment with an optional Tensorboard instance. Use the Vertex AI SDK for Python to create experiment runs. After a run has completed, use the console to visualize the run data and compare it with others. [Learn more](#)

Regionus-central1 (Iowa)

Create experiment

Filter Enter property name or value

| <input type="checkbox"/> | Name                                   | TensorBoard link                 | Updated                   | Created                   |
|--------------------------|----------------------------------------|----------------------------------|---------------------------|---------------------------|
| <input type="checkbox"/> | tuning-experiment-20251024104223519139 | <a href="#">Open Tensorboard</a> | Oct 24, 2025, 11:12:23 PM | Oct 24, 2025, 11:12:23 PM |
| <input type="checkbox"/> | tuning-experiment-20241029083920194320 | <a href="#">Open Tensorboard</a> | Oct 29, 2024, 9:09:20 PM  | Oct 29, 2024, 9:09:20 PM  |

Vertex AI

Dashboard

Model Garden

Vertex AI Studio

GenAI Evaluation

Tuning

Agent Builder

Agent Garden

Agent Engine

RAG Engine

Vertex AI Search

Vector Search

Notebooks

Colab Enterprise

Workbench

Model development

Feature Store

Datasets

Training

Experiments

pred-maint-gemini-tuned-1761327695 > Version 1

EvaluateDeploy & testBatch inferVersion detailsLineage

Deploy your model

Endpoints are machine learning models made available for online inference requests. Endpoints are useful for timely inferences from many users (for example, in response to an application request). You can also request batch inferences if you don't need immediate results.

Deploy to endpoint

| Name                                               | ID                  | Status | Models | Deployment resource pool | Region      | Monitoring | Most |
|----------------------------------------------------|---------------------|--------|--------|--------------------------|-------------|------------|------|
| <a href="#">pred-maint-gemini-tuned-1761327695</a> | 6506935101945806848 | Active | 1      | —                        | us-central1 | Disabled   | —    |
| <a href="#">pred-maint-gemini-tuned-1761327695</a> | 2647350221289291776 | Active | 1      | —                        | us-central1 | Disabled   | —    |
| <a href="#">pred-maint-gemini-tuned-1761327695</a> | 973700009767731200  | Active | 1      | —                        | us-central1 | Disabled   | —    |

Test your model

Test your model in Vertex AI studio

Open in Prompt Design

Vertex AI / Studio

New chat

Generate media

Recents

Equipment Status Classifi...

Equipment Status Classifi...

MarketMaestro: AI Lifesty...

Check GitHub Directory f...

MLops Transaction Class...

View all

Equipment EQ-001 sensor data for the last 12 hours:

- temperature\_c: Average 65.2, StdDev 6.5, Trend falling
- vibration\_hz: Average 49.9, StdDev 2.0, Trend falling
- pressure\_psi: Average 97.2, StdDev 10.0, Trend rising

Classify the equipment status based on this data (Normal, Warning, or Critical).

Status: Warning - Elevated risk detected

Evaluate

Compare

Equipment Status...

Model settings

System instructions

Give the model context to unde

Minimap

Equipment EQ-

Status: Warning - Elevated risk

## Konklúzió: A Vertex AI a specialistagyár

Végrehajtottuk a Vertex AI Blueprint-et. Egy generalista Gemini modellt egy nagy teljesítményű,

Ugrás a fő tartalomra



97,7%-os pontosságú „digitális technikussá” alakítottunk. Az igazi győzelem azonban egy megismételhető, szabályozott és skálázható folyamat létrehozása.

- **A felhőalapú tárhelyet** használtuk integrált adatbázisként.
- Hatékony hangolómotorként a beépített PEFT-tel ellátott **szerver nélküli képzési szolgáltatást** használtuk .
- Automatizált irányítási réteggént a **Modellnyilvántartást, a Kísérleteket és a Származási vonalat** használtuk .
- **A Endpoints-ot** használtuk skálázható, biztonságos telepítési motorként.

Ez a tervrajz a végleges útmutató ahhoz, hogyan lépünk túl a mesterséges intelligencia kísérletezésén és lépünk be az ipari méretű mesterséges intelligencia-termelés birodalmába.

---

## Főbb műszaki hivatkozások és további olvasmányok

1. **Vertex AI dokumentáció** : A Vertex AI összes szolgáltatásának központi központja.
2. **Bevezetés a Vertex AI Lineage-be** : Az automatizált műtermékek és végrehajtás-követés hivatalos dokumentációja.
3. **Modellek telepítése végpontra** : A modellek valós idejű előrejelzésekhez való telepítésének végleges útmutatója.
4. **LoRA: Nagy nyelvi modellek alacsony rangú adaptációja (arXiv)** : A hatékony hangolást lehetővé tevő PEFT technikáról szóló alapvető kutatási cikk.



## ✦ AI által javasolt témák

| Téma                                                                                                                                                                                                                                                                                                                                                                        | Válaszok | Megtekintések | Tevékenység |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------------|-------------|
|  Finomhangold a Llama 2-t a GCP Vertex AI-ban<br><a href="#">vertex-ai-platform</a><br><a href="#">vertex-ai-modell-nyilv...</a><br><a href="#">vertex-ai-munkapad</a>                                                                                                                     | 9        | 39            | Jun 2024    |
|  A generációs mesterséges intelligencia erejének felszabadítása: Útmutató az információkinyerés automatizálásához a Vertex mesterséges intelligenciával<br><a href="#">google-cikk</a> <a href="#">ai-ml</a><br><a href="#">legjobb gyakorlatok</a><br><a href="#">fejlesztői eszközök</a> | 4        | 34            | Jan 2024    |
| A kutatástól a gyártásig: Első betekintés az OSS által menedzselt hangolásba a Vertex AI-n<br><a href="#">google-cikk</a><br><a href="#">nyílt modellek</a> <a href="#">láma</a><br><a href="#">összehasonlító értékel...</a><br><a href="#">finomhangolás</a>                                                                                                              | 5        | 348           | Aug 28      |

© 2025 Google. Minden jog fenntartva.

[Adatvédelmi irányelvek](#)

[Szolgáltatási feltételek](#)

[Közösségi irányelvek](#)

[Visszajelzés az oldalról](#)

